Explainable AI for Effective Malware Detection and Triage

Ananta Prasad Gaire, Aleksandar Shurbevski, Shozo Naito
Kyoto College of Graduate Studies in Informatics (KCGI)
Kyoto, Japan
Emails: st052439@m2.kcg.edu, a_shurbevski@kcg.edu, s_naito@kcg.edu

***Abstract*—Security Operations Centers (SOCs) increasingly rely on machine learning to triage large volumes of potentially malicious artifacts, but high-performing models are often difficult to trust because their predictions are opaque. This paper presents an Explainable AI (XAI) pipeline for static malware detection from Windows Portable Executable (PE) files that combines strong classification performance with analyst-oriented evidence. The method uses EMBER2024-derived tabular PE features and section-level structural representations rendered as compact "section images," then fuses both modalities for malware/benign classification. On the held-out PE-only test split, our fusion model achieves ROC-AUC of 0.99795. The highest F1 score of 0.9801 is achieved together with Accuracy of 0.9802, Precision of 0.9829, and Recall of 0.9774. To improve decision transparency, the pipeline integrates SHAP for explanation of the fusion classifier and Grad-CAM for structural evidence from the CNN-based section model. We report results for a range of confidence operating thresholds, in order to reflect precision–recall trade-offs relevant to deployment. Overall, the study shows that high-performing static malware detection can be paired with interpretable evidence artifacts – model scores, ROC-based operating-point analysis, and ranked feature evidence – that support defensible SOC triage.**

I. Introduction

A. Background and Motivation

Modern organizations continuously receive, generate, and execute large volumes of digital artifacts through routine workflows such as software distribution, web browsing, and endpoint updates. In these workflows, Windows Portable Executable (PE) files are especially important from a defensive standpoint because PE is the native executable structure of Windows systems. PE files remain a dominant malware delivery format and can be distributed through common channels such as downloads, attachments, or bundled installers. At Security Operations Center (SOCs) scale, the challenge is not only detecting threats but doing so at a speed and volume that manual inspection cannot match. To handle this scale, SOCs increasingly deploy [1] machine learning (ML) systems to automate triage and prioritize suspicious files for analyst review. Deep learning models are attractive in this setting because they can learn complex interactions in high-dimensional static features that rule-based filters and signature matching often miss [2]. Yet, cybersecurity operations is a high-stakes environment: model outputs often influence decisions such as quarantining endpoints, blocking execution, or escalating incidents. These actions require accountability. If a model behaves as a black box and only provides a label (e.g., malware/benign), analysts face a trust gap: either they accept a decision they cannot justify, or they must re-investigate alerts from scratch, which reduces the operational value of automation. The trust gap grows as threats evolve and adversaries manipulate surface cues.

B. Explainable AI and Its Role in Cybersecurity

In this work, explainability is treated as evidence generation: model predictions are accompanied by feature-level and structural evidence that supports analyst verification, reporting, and triage. This is especially important in cybersecurity, where high-confidence predictions alone are often insufficient for operational decisions such as escalation, quarantine, or blocking. For the implemented pipeline, SHAP is used to explain the final LightGBM late-fusion classifier, while Grad-CAM provides complementary structural evidence for the CNN image branch of the trained section model [3], [4]. Together, these methods help analysts inspect both *which fused features mattered* and *which structural regions were emphasized*. Because explanation outputs are not automatically faithful, they must be interpreted carefully and supported by consistent preprocessing and evaluation design [5].

C. Research Aim and Scope

This research aims to develop and evaluate an explainable, proposal-aligned pipeline for static malware detection from Windows PE artifacts. The pipeline is designed to produce strong predictive performance and analyst-oriented evidence. The work focuses on static detection using the EMBER2024 PE feature set [6] and a section-level structural representation derived from executables. The scope does not include dynamic malware execution traces, malware family attribution, network intrusion detection, URL crawling, attachment sandboxing, sender reputation systems, or production SOC deployment engineering.

D. Design and Motivation

We study explainable static malware detection using EMBER2024 PE features [6]. The first modality is the tabular PE representation already provided by the dataset, while the second modality is an image embedding derived from a compact section-structure representation processed by a CNN. This design reflects the intuition that PE files contain both

statistical feature evidence and structural evidence, and that combining both may improve detection while supporting richer analyst-facing explanations. Accordingly, we compare a Tab-only baseline with a Tab+ImgEmb late-fusion model on the same held-out evaluation protocol.

We emphasize the tuned operating point because threshold selection is operationally important, while still reporting the standard threshold for comparability. Explainability is integrated in a modality-aligned manner. For the final Tab+ImgEmb LightGBM classifier, we use SHAP to generate explanation evidence over the fused feature vector, focusing on global feature rankings derived from the trained tree-based model [3]. For the image pathway, we use Grad-CAM [4] on the CNN image branch of the trained section model to generate heatmaps that highlight structurally salient regions in the section-image representation. In the final code-backed pipeline, explanation reliability depends on keeping feature layouts, exported embeddings, and model artifacts aligned across inference and analysis. We therefore enforce artifact consistency so that reported probabilities and explanation outputs remain tied to the same trained pipeline.

E. Contributions

The main contributions are:

- An explainable multimodal static malware detection pipeline using EMBER2024 PE features, including a Tab-only LightGBM baseline and a Tab+ImgEmb LightGBM late-fusion model evaluated under both standard and tuned thresholds [6].
- A multimodal explanation strategy for malware analysis, combining SHAP for the final LightGBM late-fusion classifier and Grad-CAM for structural localization in the CNN-based section model [3], [4].
- Reliability-focused implementation findings, including strict normalization alignment and feature-schema consistency for stable malware explanations.

II. Related Work

Static malware detection sits at the front line [7] of Security Operations Center (SOC) triage because it can be executed quickly, safely, and at scale, often before deeper investigation is feasible [8]. For Windows endpoints, a large share of malicious delivery still arrives as Portable Executable (PE) files [9] distributed through downloads [10], attachments, and staged droppers. Defenders must filter high-volume file streams with limited analyst time, and errors are costly. This highlights the ML "trust gap." Deep models can achieve strong predictive metrics, but SOC decisions still require justification [11]: why a file was quarantined and what evidence supports that action. Explainable AI (XAI) addresses this gap by producing interpretable signals (feature attributions and saliency maps) that connect model outputs to concrete input evidence. In cybersecurity, this evidence supports analyst validation, accelerates investigation, and reduces reliance on opaque automation. For this reason, this research treats explainability as a first-class system output and evaluates XAI behavior as part of the overall malware detection pipeline [3], [4].

A. Static Malware Detection

1) Feature-engineered PE malware detection and the EMBER line of work: A major practical tradition [12] in static malware detection is to extract specific, pre-defined properties from the Portable Executable (PE) file to create a fixed-length input vector. In the literature, these inputs are variously referred to as "handcrafted features" [13], [14], "engineered features" [12], or "tabular descriptors" [6]. To maintain consistency and precision, this paper refers to them as engineered features. These features typically include [7] byte-level statistics (e.g., histograms), entropy-related measures that can correlate with packing/obfuscation, metadata derived from headers/sections, and import/export information that reflects program behavior at a coarse structural level. The key strength of this approach is operational scalability: feature extraction and inference are fast [14], and fixed-dimensional vectors support repeatable benchmarking [6], thresholding, and controlled ablation studies [15]. The EMBER [12] dataset played a foundational role in making large-scale, reproducible PE-feature research possible by providing standardized feature extraction and evaluation conventions. EMBER2024 [6] extends this tradition by providing a large-scale, pre-computed dataset where each file is represented not as raw bytes, but as a vector of engineered values summarizing specific file characteristics. These features explicitly include: Byte-level statistics: Histograms and entropy calculations that capture the distribution of raw byte values. Header information: Metadata extracted from the COFF and Optional headers (e.g., linker versions, timestamps, and machine types). Section characteristics: Statistics regarding the size, entropy, and virtual memory layout of the executable's sections. Import and Export information: Hashed representations of imported libraries, imported functions, and exported symbols, which serve as proxies for the program's capabilities. By relying on these pre-calculated engineered features, the EMBER2024 representation enables scalable training and systematic evaluation at the feature level [6], [12]. Benchmark baselines and published results. To support reproducible comparison, the EMBER2024 authors also provide official benchmark models and runnable training/evaluation scripts in a public repository [6], [16]. These baselines are primarily LightGBM classifiers trained on EMBER feature version 3 vectors, intended to serve as reference performance levels for future work. For malicious/benign detection, the benchmark LightGBM classifier achieves ROC AUC = 0.9949 on the EMBER2024 test set [6], however, for a fixed 1% false positive rate it reaches a true positive rate of 94.48%.

2) Representation learning from raw bytes and the "end-to-end" alternative: In parallel to engineered-feature pipelines, a competing research direction [7], [17] argues that *end-to-end* models operating directly on raw bytes can discover discriminative patterns without relying on explicit feature design. MalConv [17] is a widely cited example of this

idea [13], [14], demonstrating that convolutional architectures can ingest long byte sequences to classify executables. This family of approaches has expanded the field's understanding [7], [18] of representation learning for malware, but it also highlights practical constraints that are important for SOC-facing systems: very long sequences increase compute and memory costs [14], truncation or padding choices can affect what the model learns [15], [17], and mapping byte-level signals to analyst-friendly evidence can be challenging compared to structured PE features [11], [15]. These trade-offs matter to our project design. We stay within a controlled, fixed-dimensional representation (tabular PE vectors, plus a compact "section image" representation) to keep the pipeline scalable and explanation outputs interpretable. In other words, rather than choosing between "feature engineering" and "end-to-end bytes," our work treats representation choice as an operational design decision: maintain tractable inputs that can support stable training, controlled evaluation, and explanation artifacts that analysts can act on.

3) Malware-as-image and structural visual encodings: Another influential branch [7], [19] of malware research transforms executables into image-like representations to enable CNN-based modeling. Early work [19] showed that visualizing binaries as images can support classification by capturing texture-like patterns that emerge from byte structure. More recent work [20] continues to explore image conversion techniquesnsometimes combined with additional features—showing that visual encodings can complement other representations when designed carefully. The literature further establishes that image-based approaches offer distinct advantages for interpretability and fusion. Research indicates that unlike purely tabular data, visual representations enable spatial explanation methods (e.g., Grad-CAM) that can highlight exactly which regions of a file structure influence the model's decision [4], [20]. Furthermore, studies in multimodal learning suggest that combining these structural visual signals with engineered feature vectors creates a more robust detection system, as the two modalities capture complementary patterns statistical evidence from features and spatial evidence from structure [19], [20].

4) Multimodal and fusion modeling for static malware detection: The literature increasingly suggests [7], [20] that no single static view fully captures malware behavior, especially under packing, obfuscation, and family diversity. As a result, fusion architectures that combine complementary signals have become more common. For example, research that jointly analyzes PE structure with entropy-related signals reflects the broader idea that combining structural and distributional properties can improve malware classification robustness [21]. More generally, fusion is attractive because it provides redundancy: when one modality is less informative for a sample (e.g., a heavily packed file), another modality may still retain useful evidence. This trajectory in malware research supports the hypothesis that fusing structured PE vectors with compact structural visual representations can improve detection performance without abandoning interpretability [4], [21]. By comparing tabular-only baselines against fusion architectures, recent studies aim to quantify exactly how much robustness is gained by integrating these complementary static views.

5) Explainability for malware: feature attribution for the final classifier and localization for the CNN pathway: As explainable AI becomes increasingly important in malware detection [7], [11], explanation methods must be matched to the actual model architecture used at inference time. In the implemented pipeline, the final malware decision is produced by a LightGBM late-fusion classifier rather than an end-to-end neural network. Accordingly, additive feature attribution methods such as SHAP are better aligned with the final classifier because they quantify how each fused feature contributes to the model score [3]. For malware detection, this produces ranked feature evidence that can be summarized into analyst-readable global tables, with the current implementation centered on global SHAP summaries. For CNN-based components, Grad-CAM produces class-discriminative heatmaps that highlight which spatial regions in an image-like input most strongly influenced the network response [4]. However, in the implemented pipeline, Grad-CAM is applied to the CNN image branch of the trained section model used to produce image-side evidence, while SHAP is used to explain the final LightGBM classifier. This distinction is important because it separates explanation of the final decision from supporting structural evidence in the image pathway.

A critical point in XAI literature [11], [22] especially relevant to security is that explanation outputs must be reliable, not merely visually appealing. Sanity-check research shows that some saliency or explanation methods can fail in subtle ways (e.g., remaining similar even when model parameters or labels are randomized), motivating validation procedures rather than assuming explanation faithfulness by default [23]. This supports the methodological choice to treat explanation generation and validation as part of system design rather than as a post-hoc analysis [3], [4], [23].

B. Summary and Positioning

The malware literature shows a consistent trajectory: high-performing models increasingly rely on deep learning and richer representations, but operational adoption in cybersecurity still requires transparent evidence and explanation reliability. EMBER-style PE feature benchmarks and multimodal fusion trends motivate our Tab-only baseline and Tab+ImgEmb late-fusion design, while SHAP and Grad-CAM provide explanation modalities aligned to the implemented pipeline [3], [4], [6], [12], [19], [21]. This section motivates controlled and scalable representations, multimodal fusion, and measurable explanation outputs aligned with the implemented pipeline.

III. Methodology

This paper proposes and evaluates an Explainable AI (XAI) framework for static malware detection from Portable Executable (PE) artifacts. The methodology reflects SOC requirements: high-volume detection under time constraints, with evidence for analyst validation and reporting. The project

treats explainability as a core output, not a post-hoc visualization, because security decisions require justification and traceability. The methodology emphasizes scalability and multimodal evidence, using large-scale PE feature vectors and section-derived structural representations to study whether complementary static views improve robustness. Evaluation is conducted under a consistent protocol and includes probability threshold analysis to reflect deployment-style trade-offs in security settings.

A. Malware Detection

1) Data Source and Dataset Representation: Malware detection is implemented as a static detection problem using an EMBER2024-derived PE representation [6], [16]. Each executable is mapped to a fixed-dimensional feature vector with a binary benign/malicious label, enabling scalable batch inference and feature-level explanation.

2) Preprocessing and Feature Consistency: In the final code-backed pipeline, the tabular-only model uses a fixed EMBERv3 feature layout, whereas the fusion model additionally uses exported image embeddings generated by the section model. The main implementation requirement is therefore strict feature-schema and artifact consistency across preparation, training, inference, threshold sweeping, and explanation generation. In analyst-facing settings, mismatches in feature ordering, dimensionality, or embedding generation can distort both model scores and explanation outputs.

3) Model Architectures: To evaluate the benefit of multimodal evidence while retaining interpretability, the project implements two final code-backed models: a Tab-only LightGBM baseline and a Tab+ImgEmb LightGBM late-fusion model.

The Tab-only baseline operates on the EMBERv3 tabular feature vector and predicts the malware probability directly from the fixed-length PE representation. This baseline quantifies how much discriminative power is available from the engineered static PE features alone.

The Tab+ImgEmb late-fusion model combines two sources of evidence: the EMBERv3 tabular feature vector and an exported 128-dimensional image embedding derived from the CNN image branch of a trained section model operating on section-structure representations. For each sample, the tabular feature vector is concatenated with the image embedding to form a fused feature vector, which is then passed to a LightGBM classifier to produce the final malware probability. In the released pipeline, the tabular vector has dimensionality 2568, the image embedding has dimensionality 128, and the resulting late-fusion vector has dimensionality 2696. This design preserves the complementary strengths of the two modalities: tabular PE features capture global structural and statistical signals, while the section model captures structural patterns that are summarized into the exported image embedding. Importantly for this research, the explainability pipeline is aligned to this implementation: SHAP explains the final LightGBM decision, while Grad-CAM provides complementary structural evidence from the CNN image branch of the trained section model [3], [4].

4) Evaluation Protocol and Operational Thresholding: Evaluation is performed on a held-out PE-only test split of 960,000 samples. The models output malware probabilities, which are converted to binary decisions using both the conventional threshold ($\tau = 0.50$) and a threshold selected by a sweep optimized for malicious-class F1. Reporting both operating points reflects deployment-style trade-offs between missed malware and analyst workload.

5) Explainability Methods: We use two explanation methods aligned with the implemented malware pipeline: SHAP for the final LightGBM late-fusion classifier and Grad-CAM for the CNN image branch of the trained section model [3], [4].

For the final classifier, the input to LightGBM is a late-fusion vector formed by concatenating the EMBER tabular feature vector with the exported image embedding from the section model. SHAP is applied directly to this trained tree-based model to measure how each fused feature dimension contributes to the malware score. Global explanations are produced by computing mean absolute SHAP values over an explained sample subset, yielding a ranked summary of the most influential fused features. The current implementation saves global SHAP summary artifacts for the final classifier.

Grad-CAM is not applied to the final LightGBM classifier itself. Instead, it is applied to the CNN image pathway within the trained section model used to generate image-side evidence. This produces structural heatmaps over the compact section-image representation, highlighting the regions that most strongly influence the CNN response. In this way, SHAP explains the final late-fusion decision, while Grad-CAM provides complementary structural evidence from the image side of the system.

IV. Implementation and Results

We evaluated two malware detection models: a Tab-only LightGBM baseline and a Tab+ImgEmb LightGBM late-fusion model that augments the same tabular evidence with an image embedding exported from the CNN image branch of a trained section model. The design goal is practical: scalable inference for large volumes of files with analyst-facing evidence rather than a single opaque score.

1) Decision Rule, Threshold Sweep, and Reported Metrics: For each test sample i, the model outputs a malware probability $p_i = p(y = 1 \mid x_i)$. Given a decision threshold τ, the predicted label is $\widehat{y}_i(\tau) = 1$ if $p_i \geq \tau$ and 0 otherwise. From the resulting confusion matrix counts, we compute Accuracy, Precision (malicious), Recall (malicious), and F1 (malicious). Because operational malware detection often requires balancing false negatives (missed malware) against false positives (benign files escalated), we report results at the conventional $\tau = 0.50$ and also report the best-F1 operating point identified by sweeping τ over multiple values.

2) Final Test Performance: Table I reports final performance on the full EMBER2024 PE-only test split. For the final Tab+ImgEmb model, ROC-AUC is 0.99795 and PR-AUC is 0.99805, compared with the official EMBER2024 PE-only

TABLE I
MALWARE DETECTION RESULTS ON THE EMBER2024 ALL PE FILES SET AT SELECTED OPERATING POINTS.

Model / Operating Point	τ	Acc.	Prec.	Recall	F1
Tab-only LGBM (FPR $\approx 1\%$)	0.593	0.978404	0.9898	0.9668	0.9782
Tab-only LGBM (FPR $\approx 0.5\%$)	0.823	0.972381	0.9948	0.9498	0.9717
Tab-only LGBM (FPR $\approx 0.1\%$)	0.979	0.944525	0.9989	0.8901	0.9413
Tab+ImgEmb LGBM (best-F1)	0.353	0.980175	0.9829	0.9774	0.9801
Tab+ImgEmb LGBM (FPR $\approx 1\%$)	0.593	0.978404	0.9898	0.9668	0.9782
Tab+ImgEmb LGBM (FPR $\approx 0.5\%$)	0.823	0.972381	0.9948	0.9498	0.9717
Tab+ImgEmb LGBM (FPR $\approx 0.1\%$)	0.979	0.944525	0.9989	0.8901	0.9413

TABLE II
OPERATIONAL RATES FOR THE FINAL TAB+IMGEMB MODEL ON THE FULL EMBER2024 PE-ONLY TEST SET AT SELECTED THRESHOLDS.

τ	TPR	FPR	TNR	FNR
0.353	0.9774	0.0171	0.9829	0.0226
0.500	0.9713	0.0123	0.9877	0.0287
0.593	0.9668	0.0100	0.9900	0.0332
0.823	0.9498	0.0050	0.9950	0.0502
0.979	0.8901	0.0010	0.9990	0.1099

TABLE III
SUMMARY OF EXPLAINABILITY OUTPUTS IN THE IMPLEMENTED MALWARE PIPELINE.

Method	Target	Observed role
SHAP	Final Tab+ImgEmb LGBM classifier	Provides global feature-importance evidence over the fused tabular and image-embedding input.
Grad-CAM	CNN image branch of trained section model	Highlights structurally salient regions in the section-image representation used to produce image-side evidence.

LightGBM benchmark of 0.9982 and 0.9983, respectively. At $\tau = 0.50$, the model reaches Accuracy = 0.9795, Precision = 0.9875, Recall = 0.9713, and F1 = 0.9793; at the best-F1 operating point ($\tau = 0.353$), recall rises to 0.9774 and F1 to 0.9801. The table also reports thresholds for approximate false-positive-rate regimes of 1%, 0.5%, and 0.1%, showing that stricter thresholds increase precision but reduce recall.

Figure 1 shows the ROC curve of the final Tab+ImgEmb model on the All PE files subset of the EMBER2024 dataset. The model achieves ROC-AUC = 0.99795, indicating strong class separability across operating thresholds. The final Tab+ImgEmb model also achieved PR-AUC = 0.99805, showing that precision remains high across the recall range.

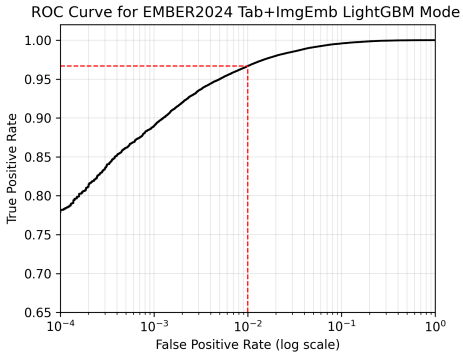

Fig. 1. ROC curve of the final Tab+ImgEmb model on the full EMBER2024 PE-only test split (ROC-AUC = 0.99795).

The main takeaway is that the fusion model improves recall and F1 while maintaining strong precision, which is important for malware triage where missed malware is typically more costly than additional analyst review.

3) Operational Interpretation: Table II summarizes operating-point behavior using rates rather than counts. As the threshold increases, FPR decreases, but TPR also decreases. At $\tau = 0.353$, the model achieves TPR = 0.9774 at FPR = 0.0171, while at $\tau = 0.500$ it reaches TPR = 0.9713 at FPR = 0.0123. At stricter thresholds such as $\tau = 0.823$ and $\tau = 0.979$, FPR falls to about 0.5% and 0.1%, respectively, but TPR also decreases. This makes threshold selection a practical choice between missed malware and analyst workload.

At $\tau = 0.353$, the model achieves TPR = 0.9774 at FPR = 0.0171, while at $\tau = 0.500$ it reaches TPR = 0.9713 at FPR = 0.0123. At stricter thresholds such as $\tau = 0.823$ and $\tau = 0.979$, FPR decreases further, but TPR also declines. This makes threshold selection a practical deployment choice between missed malware and analyst workload.

4) Explainability Report for Malware: Explainability was implemented for both parts of the malware pipeline. For the final Tab+ImgEmb LightGBM classifier, SHAP was used to generate global feature-importance evidence over the fused input vector. For the image pathway, Grad-CAM was applied to the CNN image branch of the trained section model. Thus, SHAP explains the final late-fusion decision, while Grad-CAM provides complementary structural evidence from the image pathway.

To keep the paper within the page limit, we summarize these outputs compactly rather than showing multiple heatmaps or large attribution plots. In the implemented pipeline, the explainability results support two complementary views of a malware decision: which fused features contribute most strongly in the final classifier, and which structural regions are emphasized in the section-image pathway.

These outputs improve transparency by allowing the malware predictions to be inspected beyond a single score. Although they do not establish causality, they provide useful supporting evidence for analyst-facing review.

5) What the Malware Results Mean: The malware results support three conclusions. First, the Tab+ImgEmb LightGBM late-fusion model consistently outperforms the tabular-only baseline, indicating that the image-embedding branch contributes complementary information rather than redundant noise. Second, the improvement is especially meaningful in recall: at $\tau = 0.50$, the fusion model misses fewer malicious samples, which is important because false negatives are often the more costly error in security operations. Third, the explainability outputs make the model behavior more transparent: SHAP provides feature-level evidence for the final classifier, while Grad-CAM provides supporting structural evidence from the image pathway.

V. Conclusion and Future Work

This paper presents an explainable static malware detection pipeline that combines tabular EMBER2024 features with image embeddings derived from section-structure representations. Across experiments, the Tab+ImgEmb fusion model outperformed the tabular-only baseline, with the most important operational gain being lower false negatives. On the full EMBER2024 test set (N=960,000), the Tab+ImgEmb model achieved ROC-AUC = 0.99795. At the default threshold ($\tau = 0.50$), the model reached Accuracy = 0.9795, Precision = 0.9875, Recall = 0.9713, and F1 = 0.9793. At the best-F1 operating point from the 500-threshold sweep ($\tau = 0.353$), performance improved to Accuracy = 0.9802, Precision = 0.9829, Recall = 0.9774, and F1 = 0.9801. These results support treating threshold selection as an explicit deployment policy choice rather than a fixed default.

The explainability layer improved interpretability of model behavior. SHAP was used to generate global feature-importance evidence for the final LightGBM late-fusion classifier, while Grad-CAM provided complementary structural evidence from the CNN image branch of the trained section model. In the paper, these outputs are summarized compactly to show how the malware predictions can be inspected beyond a single score.

The results should be interpreted within clear boundaries. The evaluation used a prepared split and static labels, so external validity under evolving malware distributions remains open. Feature-schema and preprocessing consistency were essential for stable probabilities and reproducible explanations, and attribution outputs should be treated as supporting evidence rather than causal proof.

Future work should prioritize deployment-oriented extensions: (i) richer semantic mapping of fused features and broader evaluation (calibration, precision–recall analysis, and time-split/family-level tests), (ii) standardized analyst evidence cards combining score, decision, top SHAP attributions, and Grad-CAM, and (iii) monitoring/governance for performance drift, explanation drift, dataset versions, feature schemas, and threshold settings.

References

[1] J. Lemon, "Sans 2025 detection and response survey: Blind spots, automation gaps, and the shift toward ai-augmented defense," SANS Institute, Tech. Rep., 2025. [Online]. Available: https://www.opswat.com/resources/reports/sans-2025-detection-response-survey

[2] B. C. Das, M. S. Sartaz, S. A. Reza, A. Hossain, M. Nasiruddin, K. K. Bishnu, K. S. Sultana, S. S. Shatyi, M. A. Khan, and J. Abed, "Ai-driven cybersecurity threat detection: Building resilient defense systems using predictive analytics," *arXiv preprint arXiv:2508.01422*, 2025. [Online]. Available: https://arxiv.org/abs/2508.01422

[3] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, 2017, pp. 4765–4774.

[4] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-cam: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017, pp. 618–626.

[5] S. Jain and B. C. Wallace, "Attention is not Explanation," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 3543–3556. [Online]. Available: https://aclanthology.org/N19-1357/

[6] R. J. Joyce, G. Miller, P. Roth, R. Zak, E. Zaresky-Williams, H. Anderson, E. Raff, and J. Holt, "Ember2024: A benchmark dataset for holistic evaluation of malware classifiers," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, pp. 5516–5526.

[7] M. Khayat, E. Barka, M. A. Serhani, F. Sallabi, K. Shuaib, and H. M. Khater, "Empowering security operation center with artificial intelligence and machine learning–a systematic literature review," *IEEE Access*, 2025.

[8] D. Gazo, A. Patel, and M. Hasan, "Effectiveness of malware incident management in security operations centres: Trends, challenges, and research directions," in *Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP)*, 2024, pp. 162–169.

[9] OpenText, "2025 opentext cybersecurity threat report," OpenText, Waterloo, ON, Canada, Tech. Rep., 2025. [Online]. Available: https://www.opentext.com/en/media/report/2025-opentext-cybersecurity-threat-report-en.pdf

[10] Recorded Future, "H1 2025 malware and vulnerability trends," Recorded Future, Somerville, MA, USA, Tech. Rep., 2025. [Online]. Available: https://www.recordedfuture.com/research/h1-2025-malware-and-vulnerability-trends

[11] N. Rastogi, S. Pant, D. Dhanuka, A. Saxena, and P. Mairal, "Measuring the security and cognitive impacts of explainability in ai-driven socs," *arXiv preprint arXiv:2503.02065*, 2025. [Online]. Available: https://arxiv.org/abs/2503.02065

[12] H. S. Anderson and P. Roth, "Ember: An open dataset for training static PE malware machine learning models," *arXiv preprint arXiv:1804.04637*, 2018.

[13] M. M.-H.-Z. Abedin and T. Mehrub, "Evaluating ensemble and deep learning models for static malware detection with dimensionality reduction using the ember dataset," *arXiv preprint arXiv:2507.16952*, 2025.

[14] E. Gandotra, D. Bansal, and S. Sofat, "Malware analysis and classification: A survey," *Journal of Information Security and Applications*, vol. 55, p. 102630, 2020, establishes that static feature extraction is computationally lighter than dynamic analysis.

[15] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3971–3988.

[16] FutureComputing4AI, "Ember2024 (repository) — examples and benchmark scripts," GitHub, 2025. [Online]. Available: https://github.com/FutureComputing4AI/EMBER2024/tree/main/examples

[17] E. Raff, J. Barker, J. Sylvester, R. Brandon, B. Catanzaro, and C. Nicholas, "Malware detection by eating a whole EXE," *arXiv preprint arXiv:1710.09435*, 2017.

[18] E. Raff, R. R. Curtin, D. Everett, R. J. Joyce, and J. Holt, "Zipf-gramming: Scaling byte n-grams up to production sized malware corpora," in *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, 2025, pp. 5988–5996.

[19] L. Nataraj, S. Karthikeyan, G. Jacob, and B. S. Manjunath, "Malware images: Visualization and automatic classification," in *Proceedings of the 8th International Symposium on Visualization for Cyber Security*, 2011.

[20] L. Zhang, T. Liu, K. Shen, and C. Chen, "A novel approach to malicious code detection using CNN-BiLSTM and feature fusion," in *2024 6th International Conference on Robotics, Intelligent Control and Artificial Intelligence (RICAI)*. IEEE, 2024, pp. 745–755.

[21] D. E. Simos, F. Zola, and F. Skopik, "Not on my watch: Ransomware detection through classification of high-entropy file segments," *Journal of Cybersecurity*, vol. 11, no. 1, p. tyaf009, 2025.

[22] J. DeYoung, S. Jain, N. F. Rajani, E. Lehman, C. Xiong, R. Socher, and B. C. Wallace, "Eraser: A benchmark to evaluate rationalized NLP models," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 4443–4458.

[23] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim, "Sanity checks for saliency maps," *Advances in Neural Information Processing Systems*, vol. 31, 2018.